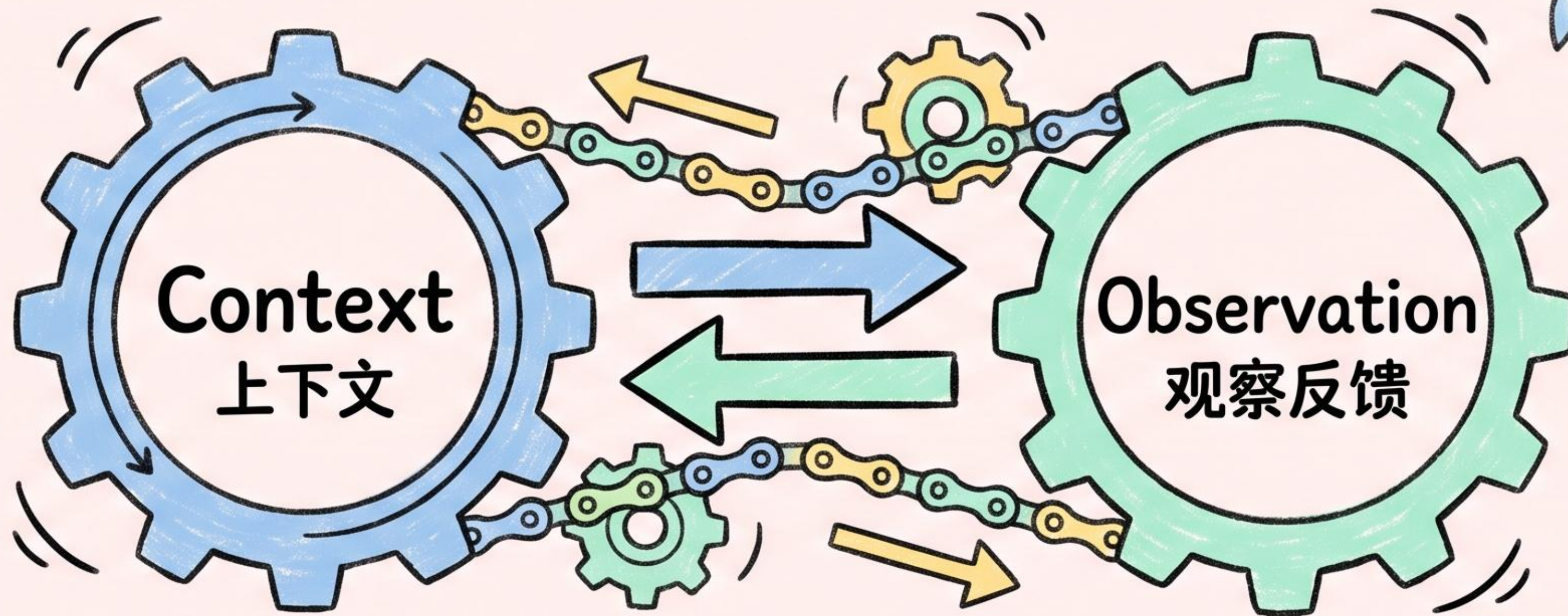


团队如何与 AI Agent 高效协作



Context × Observation 双引擎

2026 · AI Native Team Collaboration

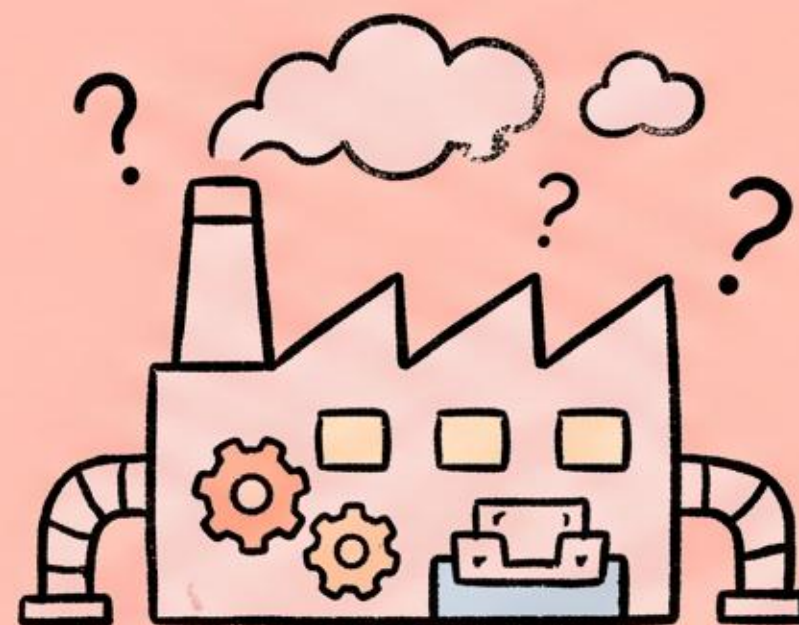
每个人都在用 AI，但团队效能提升了吗？

上下文断裂



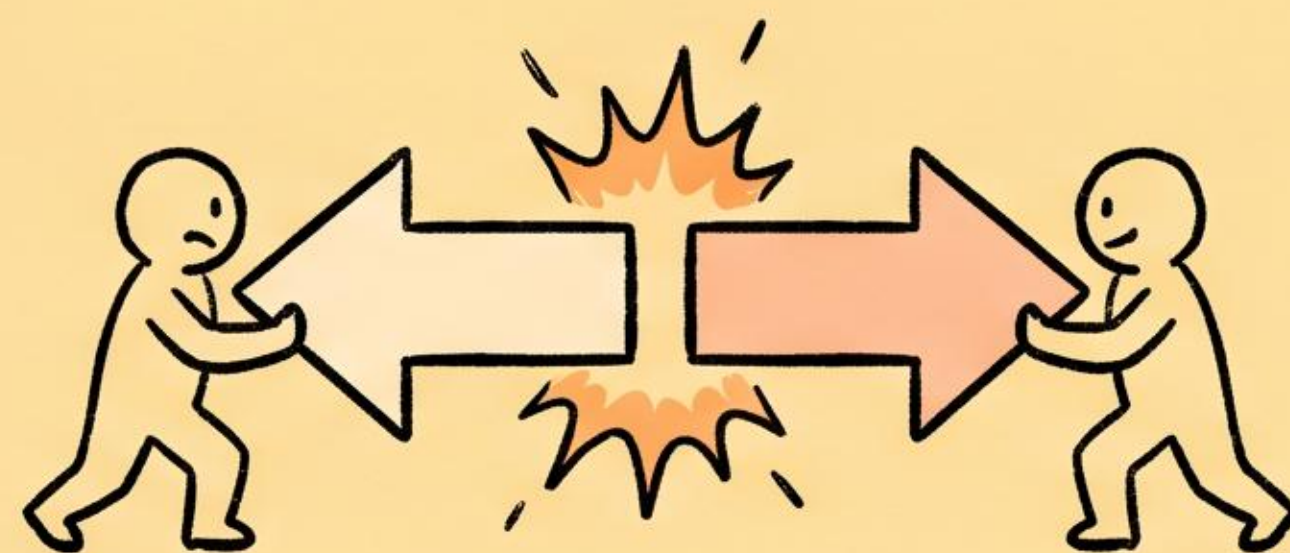
每次对话割裂、
信息无法共享

零资产沉淀



每次新建对话
AI 都是出厂设置

$1+1<2$

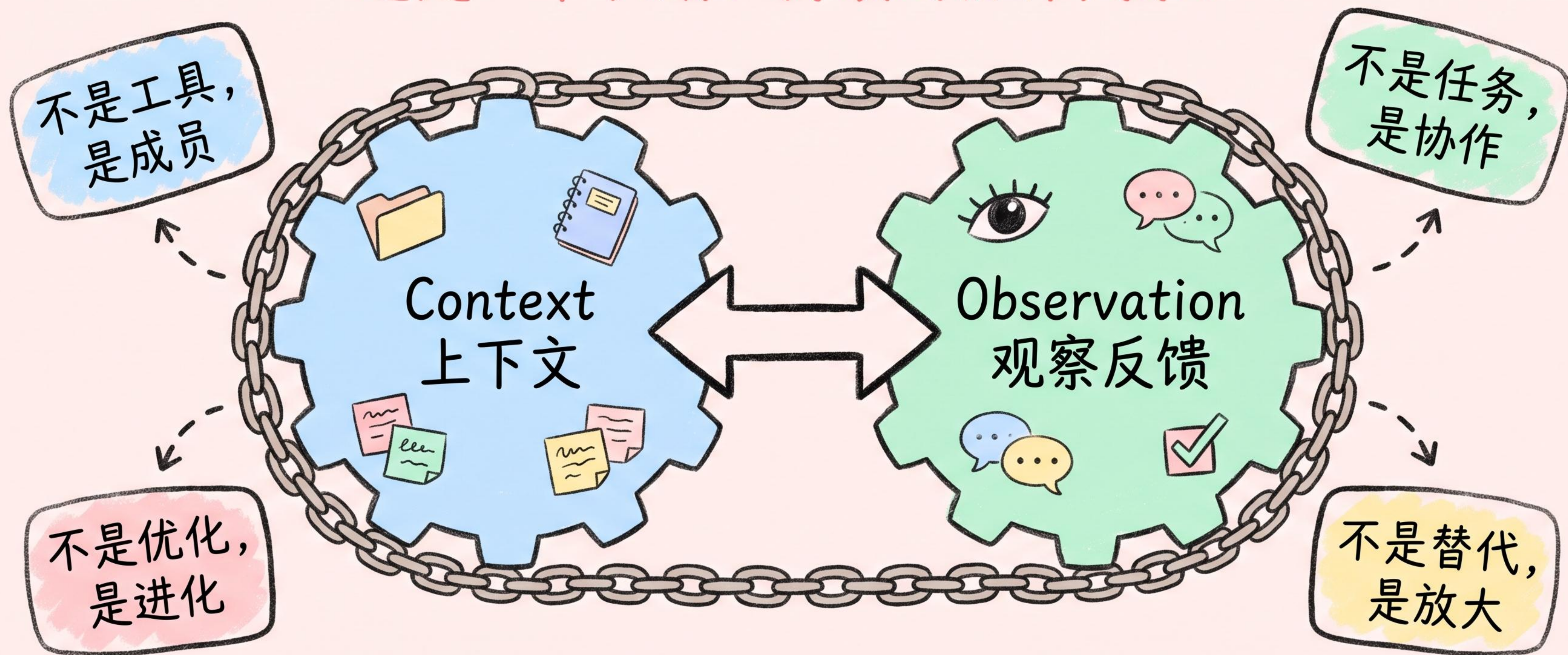


个人加法、团队摩擦

一个上午过去了，又来了一个需求。

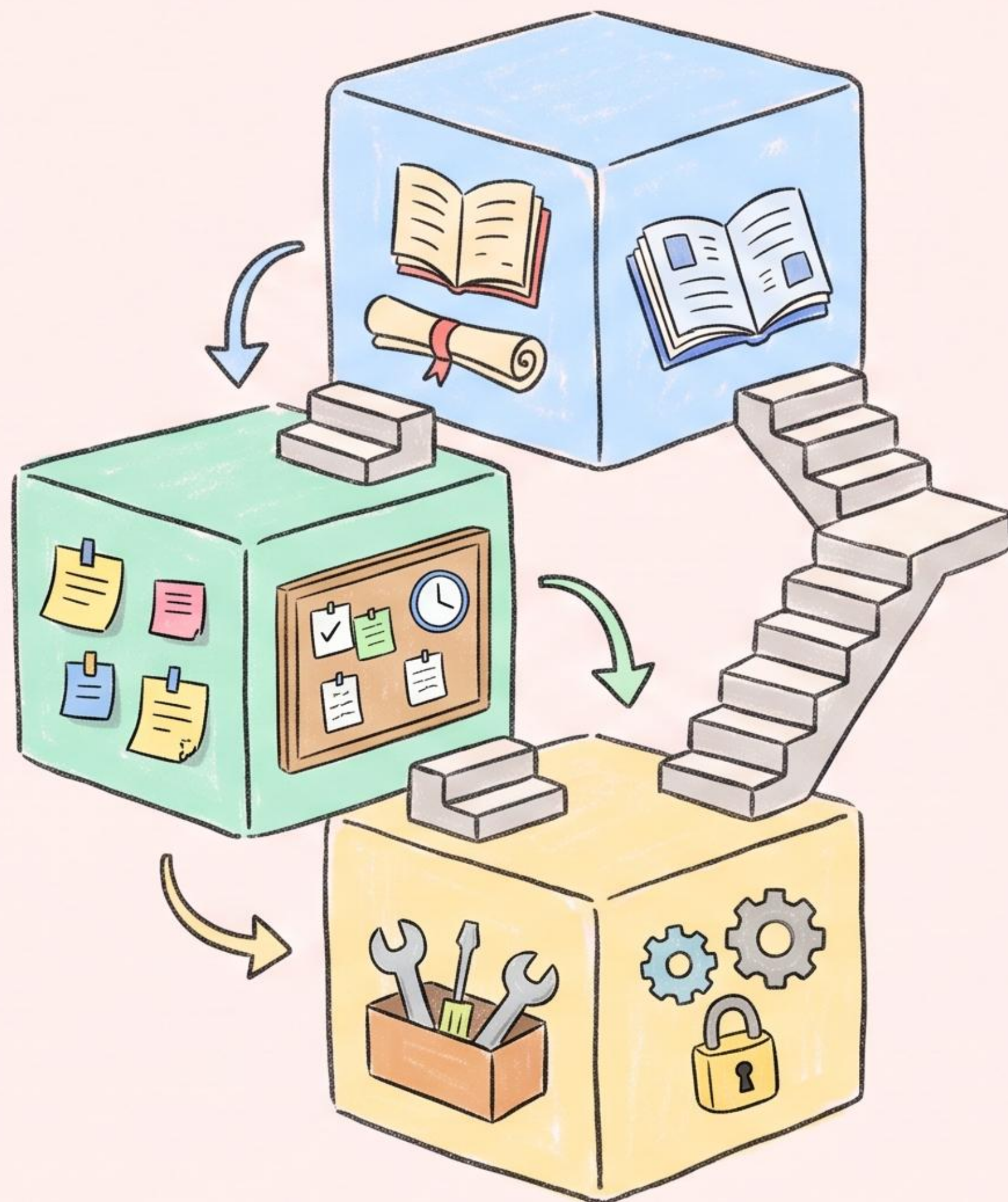


团队构建，Agent 自主执行；团队提供，Agent 持续进化。
这是一个长期、持续的协作关系。



给 AI 一个能自己做事的三层办公室

传统方式是教 AI 怎么做，“上下文工程是给 AI 一个能自己做事的环境”

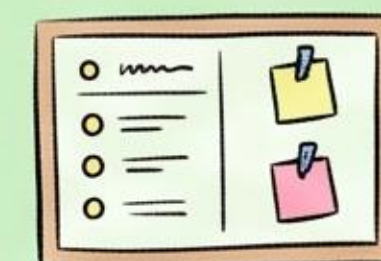


L1 静态知识库 入职手册



产品架构文档
代码规范
历史决策记录
用户画像

L2 动态工作空间 当前任务看板



会议纪要
需求优先级
代码变更记录
未关闭工单

L3 工具与权限 能干/不能干



✓ 可读可写测试
可查日志
✗ 不可部署生产
不可改用户数据

即时反馈 + 定期复盘 = Agent 的成长闭环

Agent 在 95% 情况下表现出色，Observation 要解决的就是那个 **5%**

即时反馈 Instant Feedback



合并还是退回?

采纳还是修改?



覆盖全不全?



→ 尽量让 Agent 接手那些有反馈闭环的任务

成长闭环

定期复盘 Weekly Review



任务回顾:
本周成功率多少?

错误分析:
哪些任务反复出错?



经验沉淀:
哪些场景处理得好?

→ 复盘结果 = 更新 Context + 调整约束

从人类反馈中提炼规则, 让 Agent 自主进化

团队成员的角色从执行者变成监督者和反馈者

正向反馈 Positive

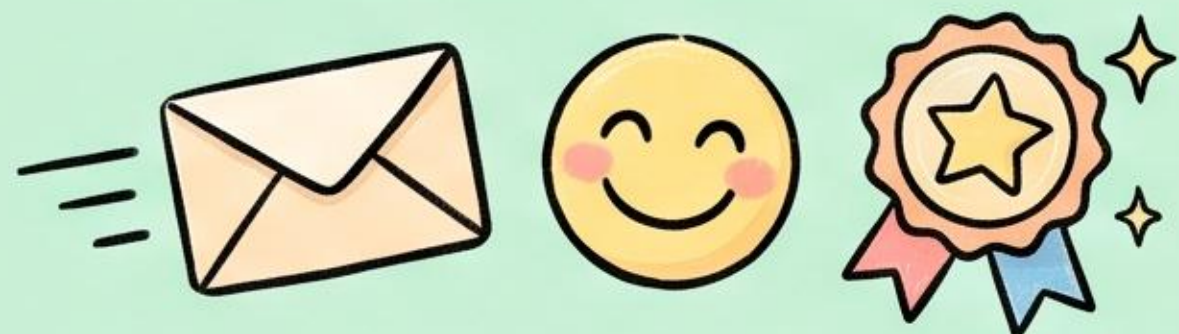


销售团队使用 Agent 生成跟进邮件, 客户回复率高

→ 团队标记优秀案例



Memory Update: 这种语气和结构的邮件转化率高



负向反馈 Negative



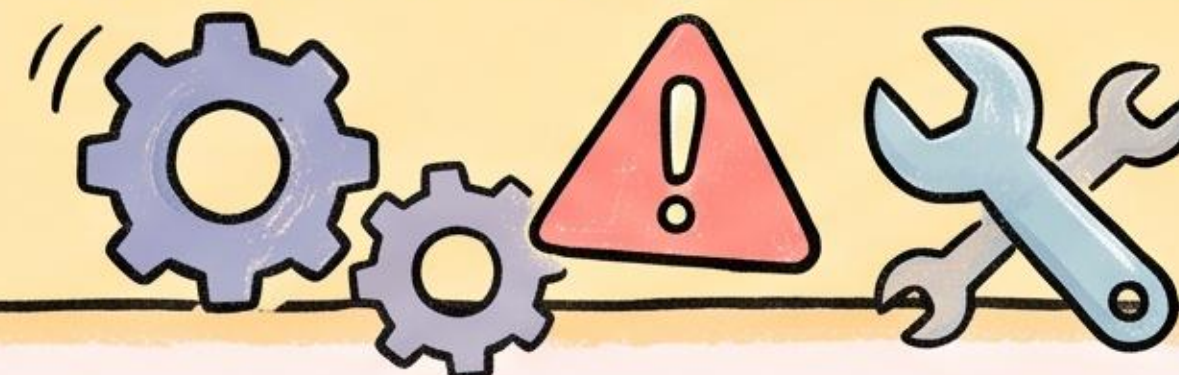
研发发现 Agent 生成的代码有性能问题

→ 修改并提交

→ Agent 捕获修改差异



Memory Update: 处理大规模并发时, 避免使用低效循环



团队的共同目标是: 养好这个 Agent。

默认所有执行工作由 AI 完成，但人的角色升级了

新的人才需要成为 Context Provider / Fast Learner / Hands-on Builder

产品经理 → Prototyper

Prototyper 原型师

从写 PRD 到 产出大量创意和原型，
定义目标和验收标准。



工程师 → Builder

Builder 构建师

从写代码 到 将原型转化为生产级产品，
构建 Context 和 Agent 基础设施。



测试 → Evaluator

Evaluator 评估者

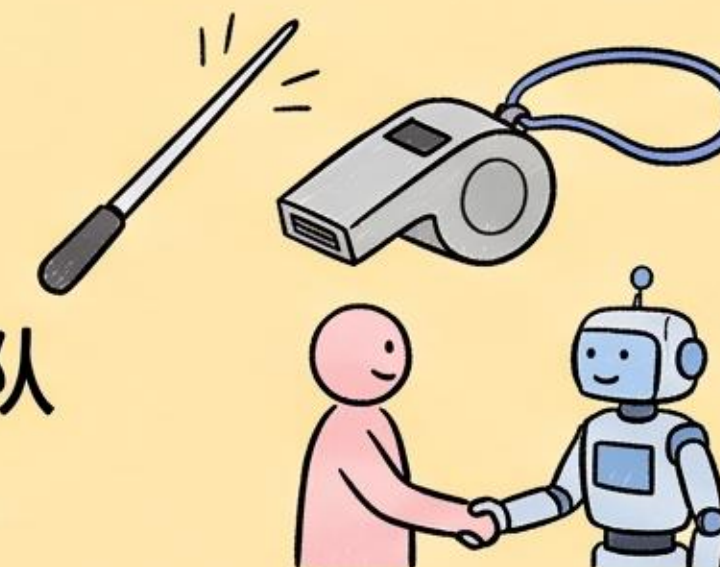
从手动测 到 设计测试策略和评估
Agent 输出，优化系统反馈机制。



管理者 → Trainer

Trainer 训练者

从管人 到 优化人机协作，治理团队
上下文，对齐 AI 系统与公司目标。

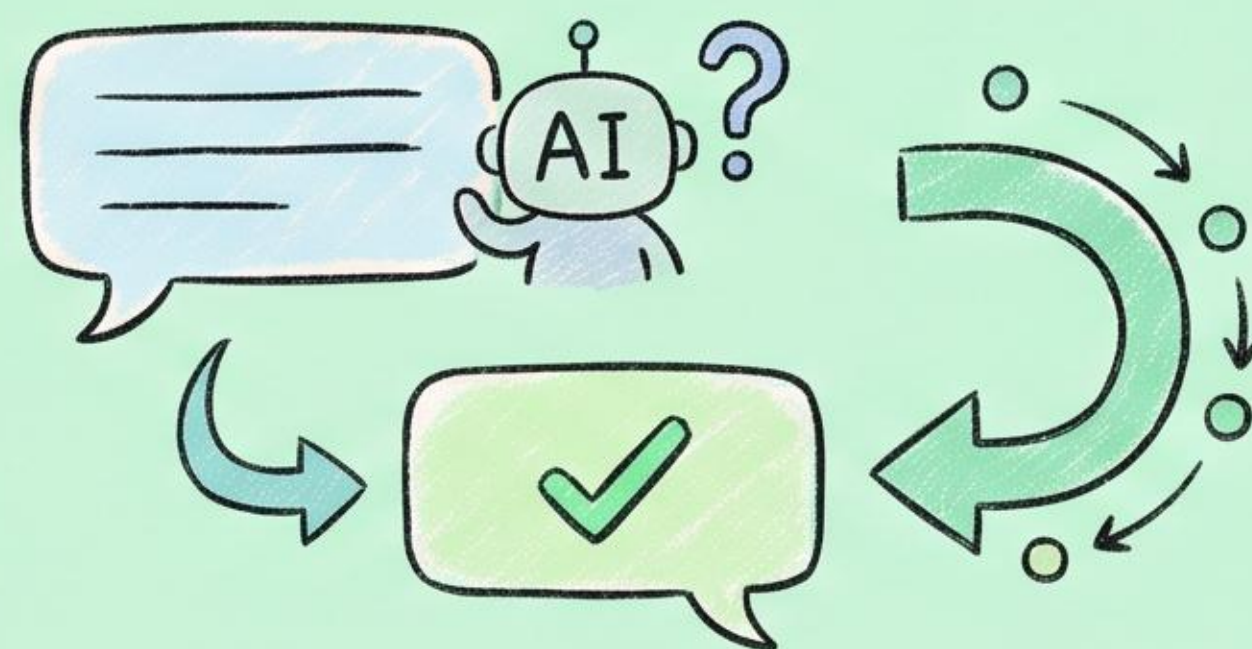


3 件今天就能开始做的事



01 文档化一切

尽可能把团队的隐性知识变成显性 Context。



02 建立反馈闭环

不要只抱怨 AI 做得不好，要把反馈作为 Observation 喂给它。



03 改变管理模式

管理要深入到团队与 Agent 的协作，而不仅是考核最终产出。

当 Context 跟着工作流走，Observation 固化进系统记忆，AI 就从打字机变成团队的 **知识中枢** —— 一个随业务一起进化的 **“大脑”**。

